Student Exercise: DML and the Basic SELECT Pipeline

SQLite + Pandas in Google Colab - no local database setup required

Scenario

You are a Data Engineer for a quick-commerce company. The analytics team receives customer, order, and payment data. Some records are messy: missing values, typo statuses, invalid customer IDs, and test/cancelled orders. Your task is to use SQL DML and SELECT queries to clean, inspect, and validate this data.

Learning Goals

• Use INSERT INTO for single-row and bulk inserts.

• Use UPDATE and DELETE safely with a WHERE clause.

• Use SELECT, DISTINCT, aliases, WHERE, AND, OR, NOT, IN, BETWEEN, LIKE, and IS NULL.

• Build a small data-quality report using SQL queries.

Exercise Overview

Part	What you will practice
A	DML: INSERT, bulk insert, UPDATE, DELETE, and safe WHERE usage
B	SELECT pipeline: specific columns, SELECT *, DISTINCT, aliases
C	Filtering: AND, OR, NOT, IN, BETWEEN, LIKE, IS NULL
D	Data quality validation using joins and NULL checks
E	Final customer revenue report for analytics

Starter Code

Run this first in Google Colab. It creates an in-memory SQLite database and sample tables.

```
import pandas as pd
import sqlite3

conn = sqlite3.connect(":memory:")
cur = conn.cursor()

def q(sql):
    return pd.read_sql_query(sql, conn)

cur.executescript("""
DROP TABLE IF EXISTS customers;
DROP TABLE IF EXISTS orders;
DROP TABLE IF EXISTS payments;

CREATE TABLE customers (
    customer_id INTEGER PRIMARY KEY,
    customer_name TEXT,
    city TEXT,
    email TEXT,
    segment TEXT
);

CREATE TABLE orders (
    order_id INTEGER PRIMARY KEY,
    customer_id INTEGER,
    order_date TEXT,
    amount REAL,
    status TEXT,
    coupon_code TEXT
);

CREATE TABLE payments (
    payment_id INTEGER PRIMARY KEY,
    order_id INTEGER,
    payment_status TEXT,
    payment_method TEXT
);

INSERT INTO customers VALUES
(1, 'Aarav', 'Delhi', 'aarav@example.com', 'premium'),
(2, 'Riya', 'Mumbai', 'riya@example.com', 'standard'),
(3, 'Anez', 'Pune', NULL, 'standard'),
(4, 'Kabir', 'Bangalore', 'kabir@example.com', 'premium'),
(5, 'Meera', 'Delhi', NULL, 'standard'),
(6, 'Ariz', 'Chennai', 'ariz@example.com', 'premium'),
(7, 'Zoe', 'Hyderabad', 'zoe@example.com', 'standard');

INSERT INTO orders VALUES
(1001, 1, '2026-01-01', 2500, 'completed', 'NEW10'),
(1002, 2, '2026-01-02', 800, 'completed', NULL),
(1003, 3, '2026-01-03', 1200, 'pending', 'SAVE5'),
(1004, 4, '2026-01-04', 0, 'complted', NULL),
(1005, 99, '2026-01-05', 1500, 'completed', NULL),
(1006, 5, '2026-01-06', 450, 'cancelled', 'TEST'),
(1007, 6, '2026-01-07', 3200, 'completed', 'VIP'),
(1008, 7, '2026-01-08', NULL, 'completed', NULL);

INSERT INTO payments VALUES
(9001, 1001, 'success', 'card'),
(9002, 1002, 'success', 'upi'),
(9003, 1003, 'pending', 'card'),
(9004, 1005, 'success', 'wallet'),
(9005, 1007, 'failed', 'card');
""")

conn.commit()
q("SELECT * FROM orders")
```

Part A - DML Tasks

Important safety rule: before every UPDATE or DELETE, first run a SELECT query with the same WHERE condition to preview the rows that will be changed.

Task	Requirement

A1	Single insert: add one new customer with name Anuz, city Jaipur, segment premium.
A2	Bulk insert: insert three new orders using executemany(). Include one completed, one pending, and one cancelled order.
A3	Update: fix the typo status 'complted' to 'completed' only for the affected row.
A4	Update: fill missing email for customer_id = 3. Do not update all customers.
A5	Delete: remove only cancelled test orders where status = 'cancelled' and coupon_code = 'TEST'.
A6	Verification: after each DML step, show the changed rows using SELECT.

Part B - SELECT Pipeline Tasks

Task	Requirement
B1	Show all rows from customers, orders, and payments using SELECT *.
B2	Select only useful order columns: order_id, customer_id, amount, status.
B3	Use DISTINCT to list unique cities, order statuses, and payment statuses.
B4	Use column aliases: amount AS order_amount, status AS order_status.
B5	Use table aliases while querying customers and orders.
B6	Create a derived column: amount * 1.18 AS amount_with_gst for completed orders.

Part C - Filtering Logic Tasks

Task	Requirement
C1	Use AND: completed orders where amount > 1000.
C2	Use OR: orders that are pending or failed/cancelled style records.
C3	Use NOT: orders where status is not completed.
C4	Use IN: customers from Delhi, Mumbai, or Chennai.
C5	Use BETWEEN: orders between 500 and 2500 amount.
C6	Use BETWEEN for dates: orders from 2026-01-02 to 2026-01-07.
C7	Use LIKE: customers whose name starts with 'A' and ends with 'z'.
C8	Use LIKE with wildcard _: customers whose name has exactly 4 characters.
C9	Use IS NULL: customers with missing email and orders with missing amount.
C10	Use IS NOT NULL: orders where coupon_code is present.

Part D - Data Engineering Validation Queries

Write SQL queries for the following checks. These are common data validation checks in ETL pipelines.

Task	Requirement
D1	Find invalid foreign keys: orders.customer_id values not present in customers.customer_id.
D2	Find successful payments for orders that are not completed.
D3	Find completed orders with NULL amount or amount <= 0.
D4	Find customers who have no orders.
D5	Create a final clean order view using only valid completed orders with valid customers and non-null positive amount.

Part E - Final Business Report

Create one final SQL report with the following columns. Use aliases to make the output readable.

```
customer_id
customer_name
city
total_completed_orders
total_revenue
average_order_value
```

Rules: include only completed orders, positive non-null amounts, and valid customers. Sort customers by total_revenue descending.

Required Interview Answers

Answer these briefly in markdown below your queries:

• What is the execution order of a SQL query? Explain FROM, WHERE, SELECT, ORDER BY in simple terms.

• How do you find all records where the name starts with 'A' and ends with 'Z' or 'z'?

• Why should NULL values be handled using IS NULL instead of = NULL?

• Why is UPDATE or DELETE without WHERE dangerous in production databases?

Submission Checklist

• A Colab notebook with all SQL queries and outputs.

• Evidence of SELECT preview before every UPDATE and DELETE.

• Completed Part A, B, C, D, and E.

• Short interview answers.

• Do not submit only final answers. The intermediate SQL outputs are required.

Grading Criteria

Area	Weight
Correct table setup and DML execution	20%
SELECT, DISTINCT, aliases, and derived columns	20%
Filtering with AND, OR, NOT, IN, BETWEEN, LIKE, IS NULL	25%
Data validation queries and final business report	25%
Interview answers and notebook clarity	10%

Helpful SQL Patterns

```
-- Preview rows before update
SELECT * FROM orders WHERE status = 'complted';

-- Safe update
UPDATE orders
SET status = 'completed'
WHERE status = 'complted';

-- Name starts with A and ends with z
SELECT * FROM customers
WHERE LOWER(customer_name) LIKE 'a%z';

-- NULL check
SELECT * FROM customers
WHERE email IS NULL;
```